The Methodology Flowchart Explained Like You're 15

A Complete, Plain-English Guide to
Every Box and Arrow in Figure 3

Thesis Defense Companion Document

November, 2026

This guide walks you through the entire experimental pipeline from top to bottom, using everyday language, real examples, and zero intimidating math.

If you can follow a recipe, you can understand this flowchart.

Contents

1 The Big Picture — What Is This Flowchart About?

Think of It Like This

Imagine you're building a system that watches sports videos and figures out what sport is being played. You can't just show a whole video to a computer — that's like dumping a 300-page book on someone's desk and saying "tell me what it's about in 2 seconds." Instead, you need a smart pipeline: pick the right pages, read them carefully, summarize what you read, and then make your decision.

That's exactly what this flowchart does, but for gymnastics videos and human action videos.

The flowchart has **8 stages** flowing from top to bottom, like water flowing downhill. Each stage takes the output of the previous stage and does something useful with it. Here they are at a glance:

Stage 1: Datasets — Where do the videos come from?

Stage 2: Frame Extraction — Pull out key snapshots from each video

Stage 3: Feature Extraction — Let a super-smart AI "look" at those snapshots

Stage 4: Feature Preprocessing — Clean and standardize the AI's notes

Stage 5: PCA Dimensionality Reduction — Shrink the notes to only the important stuff

Stage 6: Classifiers — Different "judges" that make the final decision

Stage 7: Training — How those judges learn to get better

Stage 8: Evaluation — How we score each judge's performance

Now let's zoom into each box in the flowchart, one by one.

2 Stage 1 — The Datasets (Top Two Blue Boxes)

At the very top of the flowchart, you see **two blue boxes** sitting side by side. These are your two video collections — the "raw material" that the entire pipeline feeds on.

2.1 Box 1: FineGym — 1,200 clips, 4 event classes

What is FineGym?

FineGym is a collection of **1,200 gymnastics video clips** downloaded from YouTube. Every clip shows *one* gymnast doing *one* routine on *one* apparatus. There are exactly 4 types ("classes"):

- **Balance Beam (BB)** — 300 clips
- **Floor Exercise (FX)** — 300 clips
- **Uneven Bars (UB)** — 300 clips
- **Vault (VT)** — 300 clips

That's 300 clips per class, perfectly balanced — like having exactly 300 red balls, 300 blue balls, 300 green balls, and 300 yellow balls.

Why Is FineGym Hard?

Here's the tricky part: all four categories look *almost the same*. Every single clip shows a lone gymnast in a leotard, performing acrobatic moves inside a competition arena. The background is nearly identical. The lighting is the same. The person looks similar.

Imagine being shown 4 photos of the same park at the same time of day, and being told "which photo was taken on Monday?" The differences are incredibly subtle. That's FineGym — the AI has to notice things like "is the gymnast touching a bar?" or "is there a beam beneath them?" to tell the classes apart. This is called a **fine-grained** classification problem.

Concrete Example

- **Vault clip:** A gymnast sprints down a runway, jumps on a springboard, vaults over the apparatus, and lands — all in about 5–8 seconds.
- **Balance Beam clip:** A gymnast performs jumps, turns, and tumbles on a narrow 10 cm wide beam — typically 60–90 seconds long.
- The AI must figure out which one is which, even though both show "a gymnast doing flips in a gym."

2.2 Box 2: HMDB51 — 6,754 clips, 51 action classes

What is HMDB51?

HMDB51 is a much bigger and messier collection of **6,754 video clips** covering **51 completely different human actions**. These clips come from Hollywood movies, YouTube, and old archive footage. Examples of the 51 classes include:

- **Facial actions:** smiling, laughing, chewing
- **Actions with objects:** eating, drinking, brushing hair, shooting a gun
- **Body movements:** running, climbing, jumping, doing a cartwheel
- **Human interactions:** hugging, fencing, kicking, punching

Why Is HMDB51 Different From FineGym?

HMDB51 is the *opposite* challenge. Instead of 4 similar-looking categories, you have **51 very different** categories. But the videos themselves are messy — different camera angles, different lighting, blurry footage, shaky cameras, various backgrounds.

Think of it this way:

- **FineGym** = Spotting the difference between 4 nearly identical siblings in a school photo (hard because they look alike)
- **HMDB51** = Identifying 51 different animals at a chaotic zoo during a rainstorm (hard because the conditions are terrible)

Why Use Both Datasets?

By testing on *both* datasets, we prove that our system works in two very different scenarios:

1. **Fine-grained:** Can it tell apart things that look almost identical? (FineGym)
2. **Coarse-grained:** Can it handle lots of different categories under messy conditions? (HMDB51)

If a system aces both tests, it's genuinely good — not just a one-trick pony.

3 Stage 2 — Frame Extraction (Green Box)

Both arrows from the two dataset boxes point down into a single green box labeled **"Frame Extraction."** This is where we go from videos to still images.

3.1 Why Can't We Just Feed the Whole Video?

Think of It Like This

A video is just a rapid slideshow of still pictures (called "frames"), shown so fast (usually 30 frames per second) that your eyes see smooth motion. An 8-second vault clip at 30 fps contains $8 \times 30 = 240$ individual frames — that's 240 separate photos!

Feeding all 240 frames to an AI is wasteful. Most consecutive frames are almost identical — frame 57 and frame 58 look 99% the same because only $\frac{1}{30}$th of a second has passed. It's like reading every single page of a book when you could just read every 30th page and still understand the plot.

3.2 How Frame Extraction Works

The flowchart says: **"OpenCV VideoCapture → Uniform temporal sampling → Resize to 224 × 224"**

Let's break this down into three mini-steps:

3.2.1 Step 2a: Open the Video (OpenCV VideoCapture)

What Happens

We use a software library called **OpenCV** (Open Computer Vision) — think of it as a Swiss Army knife for working with images and videos. The `VideoCapture` function opens each video file and reads it frame by frame, like opening a flipbook and being able to jump to any page you want.

3.2.2 Step 2b: Pick Evenly Spaced Frames (Uniform Temporal Sampling)

This is the most important part. Instead of grabbing all 240 frames, we pick only a *handful* of frames, spread out *evenly* across the video's timeline.

Concrete Example — A 240-Frame Vault Video, Picking 8 Frames

Imagine your video has 240 frames numbered 0 through 239. We want exactly 8 frames. So we divide the video into 8 equal chunks:

Frame #	Position in Video	Time (at 30 fps)	What's Happening
Frame 0	Very start	0.0 seconds	Gymnast standing at the start of the ru
Frame 30	$\frac{1}{8}$ through	1.0 second	Running down the runway
Frame 60	$\frac{2}{8}$ through	2.0 seconds	Approaching the springboard
Frame 90	$\frac{3}{8}$ through	3.0 seconds	Jumping onto the springboard
Frame 120	Halfway	4.0 seconds	In the air, hands on the vault table
Frame 150	$\frac{5}{8}$ through	5.0 seconds	Flying through the air, mid-flip
Frame 180	$\frac{6}{8}$ through	6.0 seconds	Starting to land
Frame 210	$\frac{7}{8}$ through	7.0 seconds	Sticking the landing

The spacing between each picked frame is $\frac{240}{8} = 30$ frames. So we pick frame 0, 30, 60, 90, 120, 150, 180, and 210. We skip frames 1–29, 31–59, etc. because they're nearly identical to the frames we already picked.

Another Example — A 150-Frame HMDB51 "Running" Video, Picking 8 Frames

Video has 150 frames. Spacing $= \frac{150}{8} \approx 19$ frames.

Frame #	Time (at 30 fps)	What's Happening
Frame 0	0.0 s	Person starting to run
Frame 19	0.6 s	Mid-stride, left foot forward
Frame 38	1.3 s	Right foot forward
Frame 56	1.9 s	Full stride in motion
Frame 75	2.5 s	Halfway through the clip
Frame 94	3.1 s	Still running, background changes
Frame 112	3.7 s	Arms pumping
Frame 131	4.4 s	Nearing end of clip

Key Insight for Defense

The word "uniform" is critical. It means *evenly spaced*, not random. We don't just grab 8 random frames — we *guarantee* that every phase of the action is represented. If we picked 8 random frames, they might all come from the first 2 seconds and miss the landing entirely!

3.2.3 Step 2c: Resize to 224 × 224 Pixels

What Happens

Every selected frame is resized (squished or stretched) to exactly **224 pixels wide × 224 pixels tall**.

Why 224? Because the AI models we use in the next step were trained on images of this exact size. It's like how a specific key only fits a specific lock — the model expects 224×224 images, so that's what we give it.

Original videos might be 1920×1080 (Full HD), 1280×720, or any other size. By resizing everything to 224×224, we make sure every video is treated the same way, regardless of its original resolution.

4 Stage 3 — Feature Extraction (Two Orange Boxes)

Below the green Frame Extraction box, two arrows branch out to **two orange boxes** — one on the left and one on the right. These are the two "AI brains" that look at the extracted frames and convert them into numbers.

Think of It Like This

Imagine you show 8 photos from a vault routine to two different expert judges:

- **Judge A (V-JEPA 2)** was trained by watching millions of videos with *no labels* — nobody told it "this is gymnastics" or "this is cooking." It learned patterns on its own, like a child learning by watching the world.
- **Judge B (Qwen2.5-VL)** was trained by looking at billions of images *with descriptions* — it was told "this photo shows a dog" or "this video shows someone cooking pasta." It learned by reading and watching together.

Both judges watch the same 8 frames, but they write down different kinds of notes because their training was completely different. We want to see whose notes are more useful for identifying actions.

4.1 Left Orange Box: V-JEPA 2 (654M)

The box says: **"V-JEPA 2 (654M) — ViT-Huge, float16+SDPA, 8 frames → 4 layers × T' × 1280"**

Let's decode this piece by piece:

What Does Each Term Mean?

- **V-JEPA 2** = The name of the AI model, made by Meta (the company behind Facebook/Instagram). "V" stands for Video, "JEPA" stands for Joint Embedding Predictive Architecture. Think of it as "Video Brain version 2."
- **654M** = The model has **654 million** adjustable knobs (called "parameters"). That's how complex its brain is. For comparison, a simple calculator has maybe a few thousand internal connections; this model has 654 million.

- **ViT-Huge** = The architecture is a "Vision Transformer — Huge size." A Transformer is a specific type of AI architecture that's great at finding relationships between different parts of an image. "Huge" just means it's the largest version.

- **float16+SDPA** = Technical memory-saving tricks. "float16" means using half-precision numbers (which take half the memory), and "SDPA" (Scaled Dot-Product Attention) is a fast way to compute attention. Think of these as "fuel-efficient mode" — same power, less memory used.

- **8 frames** = We feed it exactly 8 frames per video (the ones we extracted in Stage 2).

- **4 layers × T' × 1280** = This is the shape of its output notes:
  - **4 layers** = We collect notes from 4 different "depths" of the brain (layers 6, 12, 18, and 24 out of 24 total). Early layers notice simple things like edges and colors; deeper layers understand complex things like "a person is flipping."
  - **T'** = The number of time-steps (4 in our case, since 8 frames get grouped into 4 temporal tokens).
  - **1280** = Each note is a list of 1,280 numbers. Think of it like a very detailed survey with 1,280 questions, and each answer is a number.

What V-JEPA 2 Actually Produces

For one vault video clip, V-JEPA 2 produces:

- From Layer 6 (shallow): 4 time-steps, each with 1,280 numbers → focuses on basic shapes and textures
- From Layer 12 (mid): 4 time-steps, each with 1,280 numbers → notices body parts and apparatus shapes
- From Layer 18 (mid-deep): 4 time-steps, each with 1,280 numbers → understands poses and movements
- From Layer 24 (deepest): 4 time-steps, each with 1,280 numbers → grasps the overall action category

For **baseline classifiers**, we only use the deepest layer (Layer 24), average all the time-steps together, and get a single list of 1,280 numbers per video. Think of it as: "Just give me your final summary."

For **HAF-STD Net** (our special model), we use ALL 4 layers with all time-steps. Think of it as: "Give me your rough draft, first revision, second revision, AND final summary — I want to see how your understanding evolved."

Dashed Red Arrow — Multi-Layer Features

Notice the **dashed red arrow** coming from V-JEPA 2's left side in the flowchart. This represents the multi-layer features (all 4 layers) that go on a special path directly to HAF-STD Net, *bypassing* PCA. The regular solid arrows carry only the final-layer summary.

This is like sending the full essay draft to one teacher and just the executive summary to the other teachers.

4.2 Right Orange Box: Qwen2.5-VL (7B)

The box says: **"Qwen2.5-VL (7B) — 4-bit NF4 quantized, 4 frames → 3584-d, hidden-state pooling"**

What Does Each Term Mean?

- **Qwen2.5-VL** = An AI model made by Alibaba Cloud (the Chinese tech company). "VL" stands for Vision-Language — it understands both images *and* text together.

- **7B** = This model has **7 billion** parameters — roughly **10 times bigger** than V-JEPA 2! It's like comparing a regular library (V-JEPA 2) to the Library of Congress (Qwen2.5-VL).

- **4-bit NF4 quantized** = Because 7 billion parameters would need about 14 GB of memory (which is almost all of our GPU's 15.6 GB), we compress the model using a technique called "NF4 quantization." This is like converting a high-resolution photo to a slightly lower resolution to save phone storage — you lose a tiny bit of quality, but it still looks great, and now it fits. This shrinks the model from ∼14 GB down to ∼4 GB.

- **4 frames** = We only give it 4 frames per video (not 8 like V-JEPA 2), because the model is so much bigger that processing 8 frames would run out of memory.
- **3584-d** = Its output is a list of **3,584 numbers** per video. That's almost 3 times more than V-JEPA 2's 1,280 numbers.
- **hidden-state pooling** = We grab the model's internal "thoughts" (hidden states) from the last layer and average them all together to get one summary vector.

Important Difference Between the Two Models

Feature	V-JEPA 2	Qwen2.5-VL
Parameters	654 million	7 billion
How it learned	Watching videos *without* labels	Reading image-text pairs
Frames per video	8	4
Output size per video	1,280 numbers	3,584 numbers
Multi-layer extraction	Yes (4 layers)	No (final layer only)
Memory trick	float16 (half precision)	4-bit quantization
Runs on	Both on NVIDIA Tesla T4 (15.6 GB)	

4.3 The "Frozen" Concept — Why We Don't Retrain These Models

Think of It Like This

Both AI models are used in "frozen" mode. This means we **do not change a single one of their internal parameters**. We just use them as-is, like borrowing a pre-built telescope from NASA to look at the sky — we don't modify the telescope; we just look through it.

Why? Because these models were trained on millions (or billions) of images/videos by teams with hundreds of powerful GPUs. We only have 1 small GPU. Retraining them would be like trying to rebuild a car engine with a pocket screwdriver — not enough tools.

Instead, we just let these frozen models "look" at our videos and write down their observations (the feature vectors). Then we train much smaller classifiers on top of those observations.

5 Stage 4 — Feature Preprocessing (Green Box)

Both orange model boxes have arrows pointing down into a green box that says: **"Feature Preprocessing — StandardScaler (zero mean, unit variance) → 80/20 Train/Eval Split"**

5.1 What is StandardScaler?

Think of It Like This

Imagine you're comparing test scores from two very different exams:

- **Math exam:** scores range from 0 to 100
- **Chemistry exam:** scores range from 0 to 1000

If you just look at raw numbers, a chemistry score of 500 *looks* way bigger than a math score of 95, even though 95/100 is amazing and 500/1000 is just average. StandardScaler fixes this by converting every number so that:

- The **average** of each feature becomes **0**
- The **spread** (standard deviation) of each feature becomes **1**

Now both features are on the same playing field. A value of +2 means "2 standard deviations above average" regardless of whether it originally came from a 0-to-100 scale or a 0-to-10,000 scale.

Applied to Our Feature Vectors

V-JEPA 2 gives us 1,280 numbers per video. Some of those numbers might naturally range from −50 to +50, while others might range from −0.001 to +0.001. Without StandardScaler, the features with big numbers would dominate everything. After StandardScaler, every single one of those 1,280 features has equal influence.

5.2 What is the 80/20 Train/Eval Split?

Splitting the Data

We randomly divide all our videos into two groups:

- **80% = Training set** — The classifiers learn from these videos (like homework problems with answer keys)
- **20% = Evaluation set** — These videos are *never* shown during training. They're used only at the end to test how well the classifier learned (like the final exam)

For FineGym: 960 training clips, 240 test clips.
For HMDB51: about 5,403 training clips, 1,351 test clips.

Zero-Leakage Rule

A crucial rule: we compute the StandardScaler statistics (the average and spread) using **ONLY the training set**. Then we apply those same numbers to the test set. We **never** peek at test data to compute statistics.
Why? It's like a teacher writing an exam. If the teacher looks at students' homework answers while writing the exam, the exam would be unfairly easy. The test must be completely independent. In machine learning, breaking this rule is called "data leakage" and it would make our results meaningless.

6 Stage 5 — PCA Dimensionality Reduction (Green Box)

Below the Feature Preprocessing box, there's another green box: **"PCA Dimensionality Reduction — 95% variance retained → compact subspace ($k^* \ll D$)"**

6.1 What is PCA?

Think of It Like This

Imagine you surveyed 1,000 people with a 1,280-question questionnaire. That's a LOT of data. But many questions are redundant — if someone says they love running, they probably also said they enjoy jogging and like being outdoors. Those 3 questions essentially measure the same thing.

PCA (Principal Component Analysis) finds these redundancies and combines correlated questions into a smaller set of "super-questions" that capture the same information. If 1,280 features can be boiled down to 90 "super-features" without losing much information, PCA does exactly that.

6.2 What Does "95% Variance Retained" Mean?

The 95% Rule

"Variance" is a measure of how much useful information is spread across all your features. We keep enough "super-features" (principal components) to preserve **95% of the total information**. We're okay with losing 5% because that 5% is usually just noise — random meaningless fluctuations.

Real Numbers from the Experiments

Dataset + Model	Original Size	After PCA	Reduction
FineGym + V-JEPA 2	1,280	90	93% smaller!
FineGym + Qwen2.5-VL	3,584	142	96% smaller!
HMDB51 + V-JEPA 2	1,280	286	78% smaller
HMDB51 + Qwen2.5-VL	3,584	719	80% smaller

Notice that HMDB51 needs more components (286 and 719) than FineGym (90 and 142). That makes sense — 51 different action classes need more "descriptive power" than just 4 classes.

HAF-STD Net Skips PCA!

Look at the flowchart carefully. There's a **dashed red arrow** that goes from the Feature Preprocessing box *directly* to HAF-STD Net, **bypassing** the PCA box entirely.

Why? HAF-STD Net is specially designed to work with the full, rich, multi-layer 4D feature tensor from V-JEPA 2. Running PCA on it would destroy the temporal and layer structure that HAF-STD Net relies on. It's like trying to compress a 3D sculpture into a flat photo — you'd lose the depth information.

Only the baseline classifiers (Classical ML and Neural Networks) receive the PCA-reduced features.

7 Stage 6 — The Classifiers (Three Bottom Boxes)

Below PCA, the flowchart branches into **three boxes** — these are the "judges" that look at the processed features and make the final decision: "What action is in this video?"

7.1 Left Box (Red, Dashed): HAF-STD Net — 4.8M params

The Star of the Thesis

HAF-STD Net stands for **Hierarchical Adaptive Fusion with Spectral-Temporal Decomposition**. It's the **novel (brand new) model** proposed in this thesis — the thing that makes this research original.

It has **4.8 million** trainable parameters and is designed specifically to exploit the unique multi-layer features from V-JEPA 2. It does NOT work with Qwen2.5-VL features (because Qwen doesn't provide multi-layer temporal tensors).

How HAF-STD Net Works — The Film Critic Analogy

Imagine you're a film critic reviewing a gymnastics routine. You don't just watch the final product — you watch it at multiple levels:

1. **Spectral-Temporal Decomposition (Haar Wavelet):**
   - You split each layer's temporal signal into two streams:
     - **Appearance stream** (low-frequency): "What does the scene generally look like?" — Captures the static backdrop: the gymnast, the apparatus, the arena. Like a blurry average of neighboring frames.
     - **Motion stream** (high-frequency): "What's changing?" — Captures the differences between consecutive frames: the flips, the rotations, the speed of movement. Like the "motion blur" trail.
   - Technically, it averages pairs of neighboring time-steps (appearance) and takes their difference (motion). It's the simplest possible wavelet — called the **Haar wavelet**.

2. Depth Velocity:

- We compare what Layer 6 "saw" vs what Layer 12 "saw" vs Layer 18 vs Layer 24.
- The *difference* between consecutive layers tells us how the model's "understanding" evolved as it thought deeper. For a vault, the understanding might change dramatically between Layer 6 ("I see a person") and Layer 24 ("This is a vault with a specific twist"). For a balance beam, the evolution might follow a different pattern.
- These differences are called "depth velocities" — how fast is the understanding changing?

3. Adaptive Routing (The Smart Mixer):

- From 4 layers, we get 3 signals each (appearance, motion, velocity) = **12 streams total**.
- A small neural network looks at ALL 12 streams and decides: "For this particular video, how much should I trust each stream?" Maybe for a vault, the motion stream from Layer 18 is super important. For a floor exercise, the appearance stream from Layer 12 matters more.
- It assigns a weight (importance score) to each stream, and blends them together into one final summary.

4. Cosine Similarity Head (The Final Decision):

- Instead of the usual approach of multiplying the summary by a weight matrix, HAF-STD Net compares the summary's *direction* (not magnitude) to learned "ideal direction" vectors for each class.
- The class whose ideal direction is most aligned with the video's summary wins. It's like pointing at a compass and seeing which labeled direction you're closest to.

7.2 Center Box (Green): Classical ML (3)

Three Traditional Classifiers

These are "old-school" machine learning models that don't use neural networks:

1. **SVM-RBF (Support Vector Machine with Radial Basis Function kernel):**
   - Imagine drawing the best possible boundary line between dots of different colors on a piece of paper. SVM finds that boundary, but in 90+ dimensions instead of 2. The "RBF" part lets it draw curved boundaries instead of just straight lines.
2. **Random Forest (500 trees):**
   - Imagine asking 500 slightly different yes/no questionnaires to classify a video, then taking a majority vote. Each "tree" is one questionnaire. By averaging 500 opinions, you get a robust answer — even if some individual trees are wrong, the crowd is usually right.
3. **Logistic Regression (LogReg):**
   - The simplest possible classifier. It draws a straight line (or flat plane) between classes. It's like asking: "If I add up all the features with some weights, does the total lean more towards Class A or Class B?" Despite being simple, it often works surprisingly well.

7.3 Right Box (Cyan/Blue): Neural Network Classifiers (5)

Five Neural Network Baselines

These are small neural networks trained specifically on our features:

1. **MLP (Multi-Layer Perceptron):** Two layers of neurons (512 then 256). The basic "vanilla" neural network.
2. **DeepMLP:** A deeper version with 4 layers (512 → 256 → 128 → output),

plus Batch Normalization (keeps numbers stable) and Dropout (randomly turns off 30% of neurons during training to prevent memorization).

3. **ResMLP:** Same as DeepMLP but with "skip connections" — shortcuts that let information jump over layers. Like adding express lanes on a highway. This prevents a problem called "vanishing gradients" where deep networks forget what they learned.

4. **Conv1DNet:** Treats the feature vector as a 1D signal (like an audio waveform) and slides small filters across it to find local patterns. Good at spotting clusters of related features that are next to each other.

5. **Mini Transformer:** A tiny version of the same architecture that powers ChatGPT! It uses "attention" — the ability to look at different parts of the feature vector and figure out which parts are related to each other.

Total Classifier Count

- 3 Classical ML + 5 Neural Networks = **8 baseline classifiers**
- Plus HAF-STD Net = **9 total classifiers**

Each of the 9 classifiers is tested with features from *both* V-JEPA 2 and Qwen2.5-VL, on *both* datasets (FineGym and HMDB51). Exception: HAF-STD Net only works with V-JEPA 2 features.

8 Stage 7 — How the Neural Networks Learn (Training)

The flowchart doesn't have a separate box for training, but it's an essential invisible stage between the classifiers and the evaluation. Here's how the neural network classifiers (DeepMLP, ResMLP, Conv1DNet, Transformer, and HAF-STD Net) are trained:

8.1 The Five Training Tricks

1. Label Smoothing ($\epsilon = 0.1$)

Without label smoothing: During training, we tell the model "this video is 100% Vault and 0% everything else."

With label smoothing: We say "this video is 90% Vault, and there's a tiny 3.3% chance each that it's Balance Beam, Floor, or Bars."

Why? Because being 100% certain about anything is risky. If the model learns to be overly confident, it becomes brittle — like a student who memorizes exact answers instead of understanding the concepts. Label smoothing makes the model "humble" and better at handling unexpected inputs.

2. Feature-Space Mixup ($\alpha = 0.2$)

During training, we sometimes blend two videos' features together. For example, we might mix 70% of a vault video's features with 30% of a floor exercise video's features, and tell the model "this is 70% vault and 30% floor."

This is like a teacher creating trick questions by combining elements from different topics. It forces the model to learn *gradual* boundaries between classes instead of hard, brittle boundaries.

3. Cosine Annealing Learning Rate

The "learning rate" controls how big each learning step is. We start with big steps (learning rate = 0.001) and gradually shrink them down to tiny steps (learning rate = 0.000001), following a smooth cosine curve.

It's like learning to park a car: you start by driving toward the spot quickly, then slow down as you get closer so you don't overshoot.

4. AdamW Optimizer

The optimizer is the "engine" that actually updates the model's parameters. AdamW is a smart optimizer that:

- Adapts its step size for each parameter individually
- Uses momentum (remembers which direction it was going)
- Adds "weight decay" — a gentle pull toward zero that prevents parameters from growing too large (which would mean overfitting/memorization)

5. Early Stopping (Patience = 15)

We monitor the model's performance on a validation set after every epoch (one full pass through the training data). If the performance *hasn't improved for 15 consecutive epochs*, we stop training and roll back to the best version.

It's like baking a cake: you keep checking if it's done, and the moment it stops rising and starts burning, you take it out. "Patience = 15" means we give it 15 more chances before we decide it's done.

9 Stage 8 — Comparative Evaluation (Yellow Box at the Bottom)

All three classifier boxes have arrows pointing down into the final yellow box: **"Comparative Evaluation — Accuracy, Macro/Weighted F1, MCC, Cohen's κ"**

9.1 The Four Report Card Metrics

1. Accuracy — The Simple Score

What it means: Out of all the test videos, what percentage did the classifier get right?

Example: If there are 240 test videos and the classifier correctly identified 216, the accuracy is $\frac{216}{240} = 90\%$.

Limitation: Accuracy can be misleading if classes are imbalanced. Imagine a dataset with 95 cats and 5 dogs. A lazy model that always says "cat" gets 95% accuracy without learning anything! That's why we need the other metrics.

2. Macro F1-Score — The Fair Score (Primary Metric)

What it means: F1-Score is a blend of two things:

- **Precision:** "When you said Vault, were you right?" (avoiding false alarms)
- **Recall:** "Did you catch all the Vaults?" (avoiding missed detections)

"Macro" means we calculate F1 for *each class separately*, then take the average. This gives every class equal importance — a rare class matters just as much as a common class.

This is the primary ranking metric in the thesis. If we had to pick just one number to compare classifiers, this is it.

3. MCC (Matthews Correlation Coefficient) — The Honest Score

What it means: MCC looks at the *entire* confusion matrix — correct predictions AND incorrect predictions — and computes a correlation between predictions and truth.

- **MCC = +1.0:** Perfect predictions

- **MCC = 0.0:** No better than random guessing
- **MCC = –1.0:** Perfectly wrong (everything backwards)

The big advantage: if a model just predicts the most common class all the time, MCC gives it 0.0 ("you learned nothing"), while accuracy might give it 95%. MCC is considered the most *honest* single metric.

4. Cohen's Kappa (κ) — The Chance-Adjusted Score

What it means: Kappa asks: "How much better is this classifier than random guessing?" It adjusts the accuracy by subtracting the performance you'd expect from pure luck.

- $\kappa < 0.20$: Slight agreement (barely better than chance)
- 0.41 – 0.60: Moderate agreement
- 0.61 – 0.80: Substantial agreement
- 0.81 – 1.00: Almost perfect agreement

Why Report Four Metrics?

Each metric catches a different type of "cheating":

- **Accuracy** can be fooled by class imbalance
- **Macro F1** ensures every class is treated fairly
- **MCC** catches lazy models that only predict the majority class
- **Cohen's κ** explicitly removes the effect of random chance

If a classifier scores high on ALL four, it's genuinely good — no shortcuts, no tricks.

10 The Complete Flow — Following One Video Through the Pipeline

Let's trace a single vault video clip through every stage of the flowchart:

End-to-End Example: One Vault Video

1. **Stage 1 — Dataset:** Our clip comes from FineGym. It's an 8-second vault routine, filmed at 30 fps = 240 frames total. Its true label is "Vault (VT)."

2. **Stage 2 — Frame Extraction:** OpenCV opens the video and picks 8 evenly spaced frames: frames 0, 30, 60, 90, 120, 150, 180, 210. Each frame is resized from 1920×1080 to 224×224 pixels. We now have 8 small square images.

3. **Stage 3a — V-JEPA 2:** The 8 frames are fed into V-JEPA 2. It outputs:
   - For baselines: one vector of 1,280 numbers (final-layer mean-pooled)
   - For HAF-STD Net: a rich tensor of shape $4 \times 4 \times 1280$ (4 layers, 4 time-steps, 1280 features each)

4. **Stage 3b — Qwen2.5-VL:** 4 of the 8 frames are fed into the quantized Qwen2.5-VL. It outputs one vector of 3,584 numbers.

5. **Stage 4 — Preprocessing:** The 1,280-number vector (from V-JEPA 2) and the 3,584-number vector (from Qwen2.5-VL) are each standardized using training-set statistics. The data is split: this video lands in either the training set (80% chance) or the test set (20% chance).

6. **Stage 5 — PCA:** For baseline classifiers only: the 1,280-d V-JEPA 2 vector is compressed to 90 numbers. The 3,584-d Qwen2.5-VL vector is compressed to 142 numbers.

7. **Stage 6 — Classification:** All 9 classifiers look at the compressed (or full) features and predict a class label. SVM says "Vault." Random Forest says "Vault." LogReg says "Vault." DeepMLP says "Floor Exercise" (wrong!).

HAF-STD Net says "Vault."

8. **Stage 7 — Training:** (This already happened before testing. The neural networks trained on the 80% training set using AdamW, label smoothing, Mixup, cosine annealing, and early stopping.)

9. **Stage 8 — Evaluation:** After all test videos are classified, we compute Accuracy, Macro F1, MCC, and Cohen's Kappa for each classifier. We compare the 9 classifiers across 2 models and 2 datasets = a comprehensive comparison table.

11 Summary — The Flowchart at a Glance

The Entire Pipeline in One Paragraph

We start with two video datasets — FineGym (1,200 gymnastics clips, 4 classes) and HMDB51 (6,754 action clips, 51 classes). From each video, we extract 8 (or 4) evenly spaced frames and resize them to 224×224. These frames are fed into two frozen AI models: V-JEPA 2 (654M parameters, self-supervised, outputs 1,280-d vectors) and Qwen2.5-VL (7B parameters, vision-language, 4-bit quantized, outputs 3,584-d vectors). The resulting feature vectors are standardized (zero mean, unit variance) and split 80/20 for training and testing. For baseline classifiers, PCA compresses the features to keep 95% of the information. Nine classifiers — 3 classical (SVM-RBF, Random Forest, Logistic Regression), 5 neural networks (MLP, DeepMLP, ResMLP, Conv1DNet, Transformer), and our novel HAF-STD Net — are trained and evaluated. HAF-STD Net uniquely receives full multi-layer temporal features from V-JEPA 2, bypassing PCA, and uses wavelet decomposition, depth velocity, and adaptive routing to classify actions. All classifiers are compared using four metrics: Accuracy, Macro F1, MCC, and Cohen's Kappa.

Good luck with your thesis defense!